

LECTURER EVALUATION SYSTEM

LES

Faculty of Science | University of Ruhuna, Sri Lanka

Project Requirements, Role Definitions &
Group Member Responsibilities

Academic Year 2025 / 2026

1. Project Overview

The Lecturer Evaluation System (LES) is a web-based application developed for the Faculty of Science at the University of Ruhuna, Sri Lanka. The system digitizes and streamlines the process of collecting structured student feedback on lecturers, while providing monitoring, analytics, and reporting tools for administrative staff at multiple levels of the faculty hierarchy.

The system replaces the existing paper-based evaluation process, eliminating manual data entry, enabling real-time performance visibility, and preserving student anonymity throughout. It is designed to support five distinct user roles — Student, Lecturer, Head of Department (HoD), Dean, and System Administrator — each with a dedicated interface tailored to their responsibilities.

2. System Roles & Access Levels

The LES defines five user roles with clearly separated access rights. Role detection is automatic at login — all staff share one login page, and the system redirects each user to their appropriate dashboard based on their assigned role in the database.

2.1 Student

Students are the primary data-entry actors in the system. They log in using their university email and password and can evaluate any lecturer assigned to the modules they are enrolled in for the current semester. Their identities are never linked to their responses — all submissions are stored anonymously.

Access permissions:

- Log in with university email and password
- Filter modules by Department, Academic Year, Semester, and Course Unit
- View all lecturers assigned to a selected module
- View a lecturer's profile (photo, name, qualifications)
- Submit Theory and/or Practical feedback questionnaires
- Submit one evaluation per lecturer per module per semester (enforced by the system)
- View confirmation screen after submission

2.2 Lecturer

Lecturers log in through the Staff Login page. They can view the evaluation results for their assigned modules and access an AI-powered chatbot that summarizes the written comments left by students. They cannot see individual student responses or identify who submitted what. They are also responsible for submitting periodic supervision reports to the Admin.

Access permissions:

- View all modules assigned to them for the current semester
- View per-question evaluation results as pie charts (Strongly Agree / Agree / Neutral / Disagree / Strongly Disagree)
- Access an AI chatbot to summarize and query student comment data
- Download blank supervision report templates
- Upload completed supervision reports for Admin review
- Track the status of submitted supervision reports

2.3 Head of Department (HoD)

The Head of Department logs in via the Staff Login and is redirected to a department-level monitoring dashboard. The HoD can view aggregated evaluation performance data for all lecturers in their department, compare scores across semesters, and review individual lecturer detail including submitted reports. The HoD cannot modify evaluation data or manage system-wide settings.

Access permissions:

- View aggregated evaluation scores for all lecturers in their department
- Filter results by semester
- View a ranked list of lecturers by average evaluation score
- Drill into individual lecturer results and supervision report history
- Export department-level evaluation reports

2.4 Dean

The Dean accesses a faculty-wide analytics dashboard showing comparative performance across all departments. This role is intended for strategic oversight — identifying top performers, flagging departments or lecturers who need attention, and downloading faculty-level summary reports for accreditation or institutional review.

Access permissions:

- View faculty-wide evaluation performance overview
- Compare average scores across all departments
- View top-performing and underperforming lecturers highlighted automatically
- Drill down into any department to view individual lecturer scores
- Download faculty-level and department-level evaluation reports

2.5 System Administrator (Admin)

The Admin has the highest level of system access. They are responsible for managing all user accounts, configuring evaluation windows, managing the course and module structure, receiving and reviewing supervision reports from lecturers, and exporting data for auditing. The Admin is the only role that can create or deactivate accounts for all other roles.

Access permissions:

- Create, edit, and deactivate accounts for all user roles
- Manage departments, courses, and module–lecturer assignments per semester
- Configure evaluation windows (open/close dates per semester)
- Manage and revoke student access tokens
- Receive and update the status of supervision reports submitted by lecturers
- Export all evaluation data as PDF or CSV
- View system-wide statistics and audit logs

3. Functional Requirements

Functional requirements define the specific behaviors and features that the system must perform. They are organized by category below and numbered sequentially for traceability.

3.1 Authentication & Access Control

ID	Requirement	Priority	Role
FR01	The system shall allow students to log in using their university email and password.	High	Student
FR02	The system shall allow all staff roles (Lecturer, HoD, Dean, Admin) to log in via a shared staff login page. The role shall be automatically detected from the database and the user redirected to their appropriate dashboard.	High	All Staff
FR03	The system shall enforce role-based access control (RBAC), preventing any user from accessing pages or data outside their assigned role.	High	All
FR04	The system shall automatically expire sessions and log out users after a configurable period of inactivity.	High	All
FR05	The system shall provide a password reset mechanism via the user's registered university email address.	Medium	All

3.2 Student — Evaluation Flow

ID	Requirement	Priority
FR06	The system shall allow a student to filter available lecturers by selecting Department, Academic Year, Semester, and Course Unit from dropdown menus.	High
FR07	The system shall display a list of all lecturers assigned to the selected module. Multiple lecturers may be assigned to one module, and each must be individually evaluable.	High
FR08	The system shall display the selected lecturer's profile, including their photo, full name, department, and educational qualifications, before the student proceeds to evaluation.	High
FR09	The system shall allow the student to choose between a Theory evaluation or a Practical evaluation for the selected lecturer and module.	High

ID	Requirement	Priority
FR10	The system shall present the Theory Feedback Questionnaire, consisting of 10 items rated on a five-point Likert scale (1 = Strongly Disagree to 5 = Strongly Agree), with the Department, Course Unit, and Lecturer Name pre-filled in the form header.	High
FR11	The system shall present the Practical (Laboratory) Feedback Questionnaire, consisting of 10 items focused on lab session quality, rated on the same five-point Likert scale, with the same pre-filled header fields.	High
FR12	Both questionnaires shall include an Overall Grade field (1–5) and a mandatory written Comments field. Form submission shall be blocked and a validation message displayed if the Comments field is left empty.	High
FR13	Upon successful submission, the system shall display a Thank You confirmation screen. The system shall prevent duplicate submissions for the same student–lecturer–module–semester combination.	High
FR14	A student shall be able to evaluate multiple lecturers within a single login session without being required to log out and log in again.	Medium

3.3 Lecturer — Results & Reports

ID	Requirement	Priority
FR15	The system shall display all course modules assigned to the logged-in lecturer for the current semester on their dashboard.	High
FR16	Upon selecting a module, the system shall display the evaluation results for each of the 10 questionnaire items as individual pie charts, showing the percentage of responses in each of the five rating categories: Strongly Agree, Agree, Neutral, Disagree, and Strongly Disagree.	High
FR17	The system shall provide an AI-powered chatbot that generates a natural-language summary of the written student comments received for a selected module. The lecturer shall be able to ask follow-up questions about the feedback.	High
FR18	The system shall allow a lecturer to download a blank supervision report template in PDF format.	Medium
FR19	The system shall allow a lecturer to upload a completed supervision report. Upon upload, the report shall be automatically forwarded to the Admin for review.	Medium
FR20	The system shall display a history of the lecturer's previously submitted supervision reports, showing the title, submission date, and current status (Submitted / Under Review / Accepted).	Medium

3.4 Head of Department — Monitoring

ID	Requirement	Priority
FR21	The system shall provide the HoD with a department-level dashboard showing aggregated evaluation scores for all lecturers in their department, filterable by academic semester.	High
FR22	The system shall display a ranked list of lecturers within the department sorted by their overall average evaluation score for the selected semester.	High
FR23	The system shall allow the HoD to drill into an individual lecturer's full evaluation results, viewing per-question pie charts and a list of submitted supervision reports.	High
FR24	The system shall allow the HoD to export a department-level evaluation summary report in PDF or CSV format.	Medium

3.5 Dean — Faculty Oversight

ID	Requirement	Priority
FR25	The system shall provide the Dean with a faculty-wide dashboard displaying the average evaluation scores per department for the current semester, visualised as a comparative bar chart.	High
FR26	The system shall automatically highlight top-performing lecturers (highest average scores) and lecturers who may require attention (lowest average scores) on the Dean's dashboard.	High
FR27	The system shall allow the Dean to drill into any department to view that department's lecturer scores and individual performance details.	Medium
FR28	The system shall allow the Dean to download faculty-level and department-level evaluation summary reports in PDF or CSV format.	Medium

3.6 Admin — System Management

ID	Requirement	Priority
FR29	The system shall allow the Admin to create, edit, and deactivate user accounts for all roles: Student, Lecturer, HoD, Dean, and Admin.	High
FR30	The system shall allow the Admin to manage departments, courses, and the assignment of lecturers to course modules on a per-semester basis.	High
FR31	The system shall allow the Admin to configure evaluation windows by setting the opening and closing dates for each semester's evaluation period.	High

ID	Requirement	Priority
FR32	The system shall allow the Admin to generate and revoke student access tokens tied to specific evaluation windows.	Medium
FR33	The system shall allow the Admin to view all supervision reports submitted by lecturers, update their review status (Submitted / Under Review / Accepted), and download or view individual reports.	High
FR34	The system shall allow the Admin to export the full evaluation dataset — including all responses, scores, and comments — in PDF or CSV format for institutional auditing and accreditation.	High
FR35	The system shall provide the Admin with a system-wide statistics panel showing total users, total evaluations submitted, active evaluation windows, and pending supervision reports.	Medium
FR36	The system shall maintain an audit log of all significant system actions (logins, user creation, window configuration, report status changes) viewable and searchable by the Admin.	Medium

4. Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the system must satisfy, independent of specific user-facing behaviors. They govern how the system performs rather than what it does.

4.1 Performance

ID	Requirement	Metric
NFR01	All dashboard and questionnaire pages shall load within 3 seconds under normal network conditions.	≤ 3 seconds load time
NFR02	The system shall support a minimum of 200 concurrent authenticated users during peak evaluation periods without performance degradation.	200 concurrent users
NFR03	The AI chatbot (comment summariser) shall generate a response within 10 seconds under standard server load.	≤ 10 seconds response
NFR04	Database queries for evaluation result dashboards shall return results within 2 seconds even when the dataset contains more than 10,000 evaluation records.	≤ 2 seconds for queries

4.2 Security

ID	Requirement	Rationale
NFR05	All passwords shall be stored as salted cryptographic hashes using bcrypt or Argon2. Plaintext passwords shall never be stored or transmitted.	Prevents credential theft
NFR06	All communication between the client and server shall be encrypted via HTTPS / TLS 1.2 or higher.	Prevents interception
NFR07	Student evaluation responses shall be stored in a way that makes it impossible for a lecturer, HoD, or Dean to trace any response back to an individual student. No student identifier shall appear in the response records.	Ensures anonymity
NFR08	The system shall implement role-based access control (RBAC). Any attempt to access a page or API endpoint outside the user's assigned role shall be rejected with a 403 error and logged.	Prevents privilege escalation
NFR09	The system shall be protected against the OWASP Top 10 web vulnerabilities, including SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).	Security compliance

ID	Requirement	Rationale
NFR10	All file uploads (supervision reports) shall be validated for file type and size on the server side before being saved. Executable file types shall be rejected.	Prevents malicious uploads

4.3 Usability

ID	Requirement	Metric
NFR11	The system shall be fully responsive and functional across desktop, tablet, and mobile devices using standard web browsers.	All screen sizes
NFR12	A first-time student user shall be able to complete a full evaluation — from login to the Thank You screen — within 5 minutes without any training or external guidance.	≤ 5 minutes task time
NFR13	All form validation errors (e.g., empty mandatory Comments field) shall be displayed as clear, descriptive inline messages adjacent to the relevant field within 0.5 seconds of the user attempting to submit.	Immediate feedback
NFR14	The system shall support the English language across all interfaces. Future multi-language support (e.g., Sinhala) shall be architecturally feasible without a full rewrite.	Localization ready

4.4 Reliability & Availability

ID	Requirement	Target
NFR15	The system shall maintain an uptime of 99% or higher during active academic semesters (evaluation open periods).	99% uptime
NFR16	The system shall perform automated daily database backups. In the event of a failure, data shall be fully recoverable within 24 hours.	24-hour RTO
NFR17	If a student's session is interrupted mid-questionnaire (e.g., browser closed, network failure), no partial submission shall be saved. The student shall be required to restart the questionnaire from the beginning upon their next session.	Atomic submissions only
NFR18	The system shall handle server errors gracefully, displaying user-friendly error messages without exposing internal stack traces or database details to the end user.	Zero information leakage

4.5 Maintainability & Scalability

ID	Requirement	Rationale
NFR19	The system codebase shall follow a modular MVC (Model-View-Controller) architecture, enabling individual modules to be updated or replaced without impacting other parts of the system.	Long-term maintainability
NFR20	All API endpoints shall be documented using a standard format (e.g., OpenAPI / Swagger) so that future developers can understand and extend the system without referencing source code.	Developer handoff
NFR21	The system shall be designed to accommodate additional faculties, departments, or course structures without requiring any architectural redesign — only configuration changes.	Institutional scalability

4.6 Data Integrity & Compliance

ID	Requirement	Rationale
NFR22	The system shall enforce a one-submission-per-student-per-lecturer-per-module-per-semester rule through a database-level constraint or equivalent mechanism to guarantee data integrity.	Prevents score manipulation
NFR23	All evaluation data — questionnaire responses, scores, and comments — shall be retained for a minimum of 5 years in compliance with the University of Ruhuna's data retention policy.	Accreditation compliance
NFR24	The system shall log all data export actions (who exported, what data, at what time) in the audit trail to maintain accountability.	Compliance audit trail

5. Group Member Responsibilities

The project is divided among six group members, each responsible for a full-stack vertical slice of the system. Every member is responsible for the frontend (UI), backend (API and business logic), and database (schema and queries) related to their assigned modules. This ensures that each member gains complete end-to-end development experience and that ownership of each feature area is clearly defined.

The technology stack assumed for this division is:

- Frontend: React JS
- Backend: Node.js with a RESTful API structure
- Database: MySQL
- AI Chatbot: Integration with an LLM API (e.g., OpenAI or a local model)
- Version control: Git with GitHub, using feature branches and pull requests

Randika — Authentication & Welcome System

Member 1: Randika

Role: Authentication & System Entry Lead

Pages / Screens Responsible For:

- Welcome Page (Page 01)
- Student Sign In (Page 02)
- Staff Sign In (Page 03)
- Password Reset Flow

Frontend Responsibilities:

- Design and build the Welcome Page with three role cards (Student Sign In, Staff Sign In, Department Operator)
- Build the Student Login form (email + password, show/hide toggle, validation)
- Build the Staff Login form (shared login, same UI structure)
- Implement client-side form validation with inline error messages
- Implement session management: store JWT or session token in a secure cookie after login
- Implement the Forgot Password UI (email input, success/error feedback screen)
- Build the role-based redirect logic on the frontend (redirect to correct dashboard based on role received in login response)

Backend Responsibilities:

- Build the `/api/auth/student-login` endpoint: validate credentials, return JWT with role and user ID
- Build the `/api/auth/staff-login` endpoint: validate credentials, detect role from DB, return JWT
- Build the `/api/auth/forgot-password` and `/api/auth/reset-password` endpoints
- Implement JWT generation, signing, and verification middleware
- Implement session expiry logic (token TTL, automatic logout on expiry)
- Write authentication middleware applied globally to all protected routes
- Implement CSRF protection tokens on all login forms
- Implement rate limiting on login endpoints to prevent brute-force attacks

Database Responsibilities:

- Design the users table: id, name, email, password_hash, role (ENUM), department_id, status (active/inactive), created_at
- Design the password_reset_tokens table: token, user_id, expires_at, used
- Write SQL for credential lookup and role fetch during login
- Write SQL for token creation and expiry during password reset
- Ensure passwords are hashed with bcrypt before storage (seed scripts must never store plaintext)

Madusha — Student Evaluation Flow

Member 2: Madusha

Role: Student Evaluation Lead

Pages / Screens Responsible For:

- Student Dashboard / Course Selection (Page 04)
- Lecturer Selection (Page 05)
- Lecturer Profile View (Page 06)
- Evaluation Type Selection (Page 07)
- Thank You Screen (Page 10)

Frontend Responsibilities:

- Build the Student Dashboard with four cascading dropdown filters: Department, Academic Year, Semester, Course Unit
- Implement dynamic dropdown population (selecting Department populates Semester options, etc.) using API calls
- Build the Lecturer Selection page: grid of lecturer cards with photo, name, department badge, and Select button

- Build the Lecturer Profile page: circular photo, name, qualifications list, Start Evaluation button
- Build the Evaluation Type Selection screen: Theory and Practical choice cards with icons
- Build the Thank You screen with confirmation message and navigation options
- Implement client-side guard to prevent navigation back to a completed questionnaire (duplicate submission prevention on the frontend)

Backend Responsibilities:

- Build GET /api/departments, /api/years, /api/semesters, /api/courses endpoints with filtering logic
- Build GET /api/modules/:courseId/lecturers — returns all lecturers assigned to a module for the active semester
- Build GET /api/lecturers/:id/profile — returns lecturer photo URL, name, department, qualifications
- Build GET /api/submissions/check — checks if the current student has already submitted for a given lecturer-module-semester combination and returns a boolean
- Enforce authentication middleware on all student API endpoints

Database Responsibilities:

- Design the departments table: id, name
- Design the courses table: id, name, department_id, year, semester
- Design the lecturer_modules table: lecturer_id, course_id, semester, academic_year (linking lecturers to the modules they teach)
- Design the lecturers_profile table: user_id, photo_url, qualifications (JSON or normalized table)
- Write query to return distinct lecturers for a given module in a given semester
- Write query to check whether a submission already exists for student + lecturer + module + semester

Dulsara — Questionnaire Engine

Member 3: Dulsara

Role: Questionnaire & Submission Lead

Pages / Screens Responsible For:

- Theory Questionnaire (Page 08)
- Practical Questionnaire (Page 09)

Frontend Responsibilities:

- Build the Theory Feedback Questionnaire page: pre-filled header (Department, Course Unit, Lecturer Name), scale legend, 10 question rows each with five radio buttons (5–1), Overall Grade row, mandatory Comments textarea, Submit button
- Build the Practical Feedback Questionnaire page with the same layout and the laboratory-focused question set
- Implement client-side validation: block submission if any Likert question is unanswered, if Overall Grade is not selected, or if Comments is empty; show inline error messages
- Implement submission state (disable Submit button after click to prevent double-click submissions)
- Render all 10 question texts and the overall grade field dynamically from an API response so that questions can be updated from the backend without code changes
- Display a loading spinner during form submission and redirect to Thank You screen on success

Backend Responsibilities:

- Build POST /api/submissions — accepts the full questionnaire payload (type: theory/practical, lecturer_id, module_id, semester, responses: [{question_id, score}], overall_grade, comment) and saves it atomically
- Validate all inputs server-side independently of frontend validation: required fields, score range (1–5), comment not empty
- Enforce the one-submission rule server-side by checking the submissions table before inserting; return 409 Conflict if duplicate detected
- Store the comment without any student identifier to preserve anonymity
- Build GET /api/questions — returns the full list of Theory and Practical questions with their IDs and text
- Build the questions table with seeded data so questionnaire content is database-driven

Database Responsibilities:

- Design the questions table: id, type (theory/practical), label (a-j), question_text, display_order
- Design the submissions table: id, type, lecturer_id, module_id, semester, academic_year, overall_grade, submitted_at — NO student_id column (anonymity)
- Design the submission_responses table: id, submission_id, question_id, score (1–5)
- Design the submission_comments table: id, submission_id, comment_text — stored separately for easy AI processing
- Write atomic INSERT transaction for submissions + responses + comment
- Write the duplicate-check query: COUNT(*) WHERE type + lecturer_id + module_id + semester matches a session-tracked student submission token
- Seed the questions table with all 10 Theory questions and all 10 Practical questions

Sujithran — Lecturer Dashboard & AI Chatbot

Member 4: Sujithran

Role: Lecturer Dashboard & AI Integration Lead

Pages / Screens Responsible For:

- Lecturer Dashboard (Page 11)
- Evaluation Results View with pie charts (Page 12)
- AI Chatbot panel (within Page 12)

Frontend Responsibilities:

- Build the Lecturer Dashboard showing a grid of module cards for the current semester (module code, name, semester badge, View Evaluation button)
- Build the Evaluation Results page: for each of the 10 questions, render a pie chart showing the percentage breakdown of the five rating categories using Chart.js or a similar library
- Display an Overall Grade Distribution chart (full-width) at the bottom of the results page
- Build the AI Chatbot panel: a scrollable chat bubble area showing the AI-generated summary and a text input for follow-up questions
- Show a loading indicator while the AI summary is being generated
- Handle chatbot error states gracefully (e.g., if the API is unavailable, show a friendly fallback message)
- Implement the Supervision Reports section on the dashboard: list of submitted reports with status badges, Download Template button, Upload button with drag-and-drop zone

Backend Responsibilities:

- Build GET /api/lecturer/modules — returns modules assigned to the authenticated lecturer for the current semester
- Build GET /api/lecturer/evaluation-results/:moduleId — returns per-question score distributions (count of each rating per question) and overall grade distribution
- Build POST /api/ai/summarize — accepts module_id, fetches all comments for that module from the database, sends them to the AI API (e.g., OpenAI GPT-4), and returns a natural-language summary
- Build POST /api/ai/chat — handles follow-up questions from the lecturer by maintaining conversation context and calling the AI API
- Build GET /api/lecturer/supervision-reports — returns the lecturer's submitted reports with status
- Build POST /api/lecturer/supervision-reports — handles file upload (validate file type: PDF/DOCX only, max 10 MB), saves to storage, records in DB, notifies Admin

- Build GET /api/supervision-template — serves the blank supervision report template file

Database Responsibilities:

- Design the supervision_reports table: id, lecturer_id, file_path, file_name, submitted_at, status (submitted/under_review/accepted), reviewed_by (admin user_id, nullable), reviewed_at
- Write aggregation query for pie chart data: `SELECT question_id, score, COUNT(*) FROM submission_responses WHERE submission_id IN (...) GROUP BY question_id, score`
- Write query to fetch all comments for a given module and semester for the AI summarizer
- Write query for overall grade distribution: `SELECT overall_grade, COUNT(*) FROM submissions WHERE ... GROUP BY overall_grade`
- Set up file storage path conventions and ensure uploaded files are saved outside the web root for security

Ijini — HoD, Dean & Supervision Report Review

Member 5: Ijini

Role: Analytics & Monitoring Dashboard Lead

Pages / Screens Responsible For:

- HoD Department Dashboard (Page 14)
- HoD Lecturer Detail View (Page 15)
- Dean Faculty Overview (Page 16)
- Dean Department Drill-Down (Page 17)
- Lecturer Supervision Report page (Page 13)

Frontend Responsibilities:

- Build the HoD Dashboard: 4 KPI cards (Total Lecturers, Total Evaluations, Average Score, Pending Reports), sortable lecturer table (name, modules, Theory score, Practical score, overall score, reports submitted), semester filter dropdown
- Build the HoD Lecturer Detail View: lecturer profile card, pie chart results grid (same component as Page 12 but in read-only mode), supervision report list with download links
- Build the Dean Faculty Overview: 4 KPI cards, grouped bar chart comparing all departments' average scores, top performers section and needs-attention section
- Build the Dean Department Drill-Down: triggered by clicking a department on the bar chart; shows that department's lecturers sorted by score with a back button
- Build the Lecturer Supervision Report page (Page 13): Download Template button, drag-and-drop upload zone, submitted reports history table

- Implement chart interactivity (click a bar to drill down, hover for tooltips) using Chart.js
- Implement PDF/CSV export buttons on HoD and Dean dashboards

Backend Responsibilities:

- Build GET /api/hod/department-overview — returns aggregated scores for all lecturers in the HoD's department for a given semester
- Build GET /api/hod/lecturer/:id/details — returns full results and supervision report list for one lecturer
- Build GET /api/dean/faculty-overview — returns average scores per department for all departments in the faculty
- Build GET /api/dean/department/:id — returns individual lecturer scores within a department
- Build GET /api/export/department-report and /api/export/faculty-report — generates and streams a PDF or CSV report using a server-side library (e.g., PDFKit or pdfmake for PDF, json2csv for CSV)
- Ensure all HoD and Dean endpoints verify the requester's role and department membership before returning data

Database Responsibilities:

- Write department-level aggregation query: average per-question scores and overall average for each lecturer within a department for a given semester
- Write faculty-level aggregation query: average overall score per department across all lecturers and submissions
- Write top-performer query: SELECT top 3 and bottom 3 lecturers by average overall score faculty-wide
- Ensure all analytics queries use indexed columns (semester, department_id, lecturer_id) for performance
- Write query to count pending supervision reports per department for the HoD KPI card

Hasna — Admin Panel & System Configuration

Member 6: Hasna**Role: Admin Panel & System Configuration Lead****Pages / Screens Responsible For:**

- Admin Overview Dashboard (Page 18)
- User Management (Page 19)
- Course & Module Management (Page 20)
- Evaluation Window Management (Page 21)

- Reports & Audit Log (Page 22)

Frontend Responsibilities:

- Build the Admin Overview Dashboard: 6 KPI cards (Total Students, Total Lecturers, Total HoDs, Total Submissions, Active Evaluation Windows, Pending Supervision Reports), submission trend line chart, Recent Reports and Recent Activity side-by-side panels
- Build the User Management page: searchable and filterable table of all users (ID, Name, Email, Role badge, Department, Active/Inactive toggle), Add New User floating button, Add User modal form (Name, Email, Role, Department, Save/Cancel)
- Build the Course & Module Management page: accordion tree of departments/courses on the left, Module–Lecturer Assignment table on the right with per-row lecturer dropdown
- Build the Evaluation Window Management page: table of semesters with open/close date configuration, Add Window modal with date pickers, Token Management section with Generate and Revoke actions
- Build the Reports & Audit Log page: filter bar for data export (Role, Department, Semester, Date Range, Export PDF/CSV buttons), Supervision Report Inbox table with status update dropdown, Audit Log scrollable table with search and date filter
- Implement all modals as non-fixed overlays (positioned within the page flow) with backdrop, Save/Cancel, and error state handling

Backend Responsibilities:

- Build full CRUD API for users: GET /api/admin/users, POST /api/admin/users, PUT /api/admin/users/:id, PATCH /api/admin/users/:id/deactivate
- Build CRUD API for departments and courses: GET, POST, PUT, DELETE /api/admin/departments and /api/admin/courses
- Build API for module–lecturer assignment: POST /api/admin/module-assignments, DELETE /api/admin/module-assignments/:id
- Build API for evaluation windows: GET, POST, PUT /api/admin/evaluation-windows; enforce that windows cannot overlap for the same semester
- Build API for supervision report status updates: PATCH /api/admin/supervision-reports/:id/status
- Build data export endpoints: GET /api/admin/export/evaluations with query params for filtering; stream generated PDF or CSV to the browser
- Build GET /api/admin/audit-log with pagination and search; write a reusable audit logging utility function called by all other admin endpoints
- Build GET /api/admin/dashboard-stats returning all 6 KPI counts in a single efficient query

Database Responsibilities:

- Design the evaluation_windows table: id, semester, academic_year, open_date, close_date, created_by

- Design the audit_log table: id, user_id, action, entity_type, entity_id, details (JSON), timestamp
- Write the dashboard stats query: a single SQL query (or stored procedure) returning all 6 KPI values using subqueries or CTEs for efficiency
- Write queries for data export with parameterized filters (semester, department, role, date range)
- Ensure all admin write operations INSERT into audit_log within the same transaction
- Set up database indexes on foreign keys (user_id, department_id, semester, submitted_at) used in admin queries
- Write database seed scripts for initial Admin account, sample departments, and sample courses

6. Shared Group Responsibilities

In addition to their individual module responsibilities, all six group members share the following collective obligations:

Area	Shared Responsibility
Database schema design	All members must jointly finalize the complete database schema before development begins. Schema changes that affect another member's module must be communicated and agreed upon before implementation.
Git workflow	All members must use feature branches (e.g., feature/auth, feature/student-flow) and submit pull requests for code review before merging into the main branch. Direct commits to main are not permitted.
API contract	All members must agree on and document the API request/response formats (endpoint URLs, parameter names, response structures) before frontend development begins, to prevent integration conflicts.
Security practices	All members are individually responsible for applying input validation, parameterized queries, and authentication middleware to every endpoint they build.
Testing	Each member must write basic test cases (unit or integration) for the backend endpoints they develop. Manual UI testing of their own pages is also required before submitting a pull request.
Code review	Each pull request must be reviewed and approved by at least one other group member before merging.
Documentation	Each member must document the API endpoints they build (route, method, request body, response format) in the shared API documentation file.
Final integration	The final week is reserved for joint integration testing — connecting all six modules together and resolving cross-module issues as a team.

7. Recommended Technology Stack

Layer	Technology	Rationale
Frontend	React.js (or plain HTML/CSS/JS)	Component reusability across role dashboards; easy state management for multi-step forms
Styling	Tailwind CSS or Bootstrap 5	Rapid responsive UI development; mobile-first design
Charts	Chart.js	Pie charts for evaluation results; bar charts for HoD/Dean dashboards; lightweight

Layer	Technology	Rationale
Backend	Node.js with Express.js (or PHP Laravel)	RESTful API structure; large ecosystem; easy JWT integration
Database	MySQL 8 or PostgreSQL 14	Relational model suits the structured evaluation data; strong foreign key support
Authentication	JWT (JSON Web Tokens) + bcrypt	Stateless role-based auth; bcrypt for secure password hashing
File Storage	Local server storage or AWS S3	For supervision report PDF uploads; S3 preferred for scalability
AI Chatbot	OpenAI API (GPT-4o) or Google Gemini API	Natural-language summarization of student comments; follow-up Q&A
PDF Export	PDFKit (Node) or FPDF (PHP)	Server-side generation of evaluation and supervision report PDFs
Version Control	Git + GitHub	Feature branch workflow; pull request reviews
Hosting	University server or shared hosting (Apache/Nginx)	Deployment within the university's infrastructure